

Scrutiny: Mühendislikte Derinlemesine İrdeleme Pratiği

Ömer Faruk Yavuz

August 2026

Kavramın Tanımı ve Terminolojik Çerçeve

Bu çalışmanın konusu, bir nesnenin, sistemin veya iddianın yüzeysel biçimde gözden geçirilmesi değil; en küçük ayrıntısına kadar sistematik olarak sorgulanması pratiğidir. Literatürde bu edim için tek bir karşılık bulunmamakla birlikte, en kapsayıcı terim *scrutiny* (fil hâliyle *to scrutinize*) olarak kabul edilebilir. Türkçe akademik yazımda buna en yakın karşılıklar *irdeleme*, *derinlemesine inceleme* ve *ince eleyip sık dokuma*, *DİDİKLEME* ifadeleridir.

Söz konusu edimin temel bileşenleri şunlardır:

- Görünür olmayanın açığa çıkarılması
- Gizli kalmış hataların araştırılması
- Varsayımların tek tek sınanması
- Hiçbir önermenin gerekçesiz biçimde doğru kabul edilmemesi

JPL, NASA ve SpaceX gibi kurumlarda yürütülen tasarım incelemeleri “Bu tasarım hangi koşulda başarısız olur?” sorusu üzerine kuruludur. Amaç tasarımın savunulması değil, dayanamayacağı sınır noktalarının tespitidir.

Bu yaklaşım, epistemik alçakgönüllülük ilkesiyle doğrudan ilişkilidir: yetkin mühendis çalışmasına “tasarımım kusursuzdur” önermesiyle değil, “hangi noktada yanılıyor olabilirim?” sorusuyla başlar.

Yöntemin normatif özeti şu biçimde formüle edilebilir: iyi bir tasarım inceleme, tasarımı savunmak amacıyla değil, onu çürütebildiği ölçüde çürütmeye çalışmak amacıyla yürütülür. Çürütme girişimi başarısız olduğunda tasarıma duyulan güven artar.

İrdeleme Katmanları

İrdeleme tek bir düzlemde gerçekleştirilmez; aynı nesneye farklı analitik merceklerle yinelemeli olarak bakılması gerekir.

1. **Amaç katmanı:** Sistem hangi problemi çözmeyi hedefliyor? Yanlış problem mi ele alınıyor?
2. **Gereksinim katmanı:** Talep edilen doğru tanımlanmış mı? Eksik, çelişkili veya ölçülemez gereksinim mevcut mu?
3. **Varsayım katmanı:** Hangi önermeler doğru kabul edildi? Hangi varsayımın geçersizleşmesi tasarımı çökertir?
4. **Mimari katman:** Bileşen ayrıştırması isabetli mi? Bağımlılıklar, sınırlar ve darboğazlar nerede konumlanıyor?
5. **Arayüz katmanı:** Sistemlerin temas yüzeyleri; veri biçimi, zamanlama, protokol, fiziksel bağlantı ve insan-makine etkileşimi. Hataların önemli bir bölümü bu katmanda ortaya çıkar.
6. **Bileşen ve uygulama katmanı:** Kod, devre, mekanik parça, algoritma ve malzeme seçimleri doğru mu?
7. **Veri ve durum katmanı:** Veri hangi kaynaktan geliyor? Bayat, eksik, yanlış sıralı veya çelişkili olma ihtimali var mı? Sistem kendi durumunu doğru biliyor mu?
8. **Zaman katmanı:** Doğru işlem yanlış zamanda gerçekleşirse ne olur? Race condition, gecikme, senkronizasyon, timeout ve geçici rejim sorunları.
9. **Hata katmanı:** Neler bozulabilir? Tekil hata mı yeterli, çoklu hata mı gerekli? Sistem fail-safe mi, fail-operational mı?
10. **Sınır koşulları katmanı:** Minimum, maksimum, aşırı yük, düşük ve yüksek sıcaklık, düşük güç, yoğun trafik ve sıra dışı kullanıcı davranışı.
11. **Güvenlik ve emniyet katmanı:** Sistem istem dışı zarar üretebilir mi? Kötü niyetli aktör sistemi nasıl istismar edebilir?
12. **İnsan katmanı:** Operatör hangi noktayı yanlış yorumlayabilir? Bakım personeli hatalı parça takabilir mi? Arayüz hataya davetiye çıkarıyor mu?
13. **Üretim ve bakım katmanı:** Tasarım üretilebilir, test edilebilir, onarılabilir mi? Yedek parça tedariki sürdürülebilir mi?
14. **Kanıt katmanı:** “Çalışıyor” iddiasının dayanağı nedir? Test, analiz, simülasyon veya saha verisi mevcut mu? Kanıtın niteliği yeterli mi?
15. **Karar gerekçesi katmanı:** A seçeneği neden B yerine tercih edildi? Bu karar hangi koşulda yeniden değerlendirilmelidir?

16. **Bilinmeyenler katmanı:** Neyi bilmediğimizin farkında mıyız? Hangi konu henüz ölçülmedi veya sınanmadı?
17. **Sistemler arası etki katmanı:** Bu değişiklik başka hangi sistemi etkiler? İkincil ve üçüncül derece sonuçlar nelerdir?
18. **Operasyon katmanı:** Gerçek kullanım koşullarında ne gerçekleşir? Kullanıcılar tasarımı öngörüldüğü biçimde mi kullanacak?
19. **Yaşam döngüsü katmanı:** Bir yıl, on yıl sonrası nasıl görünüyor? Güncelleme, eskime, veri göçü, teknik borç ve tedarikçi sürekliliği.
20. **Kurumsal katman:** İlgili bilgi tek bir kişide mi toplanmış? Bu kişinin ayrılması hâlinde hangi bilgi kaybedilir?

Ekiplerin büyük bölümünün ihmal ettiği ilave bir katman bulunmaktadır: incelemenin kendisinin incelenmesi.

- Doğru kişilere mi danışıldı?
- Sorulan soru doğru soru muydu?
- Uygulanan test, aranan hatayı tespit edebilecek nitelikte miydi?
- Ekibin tamamı aynı kör noktayı paylaşıyor olabilir mi?

Yüksek olgunluk düzeyine sahip mühendislik kültürü tam bu noktada başlar: yalnızca sistemin değil, sistem hakkında düşünme biçiminin de irdelenmesi.

Fiziksel Düzlemde İrdeleme

Fiziksel irdeleme; parçanın elle, gözle, ölçüm cihazıyla ve kontrollü zorlama yoluyla sınanmasıdır.

Uygulanan İşlemler

- Konnektörlerin tek tek hareket ettirilmesi: gevşeklik tespiti
- Kablonun farklı açılarda bükülmesi: temas kesilmesi kontrolü
- Vida torkunun doğrulanması
- Büyüteç veya mikroskop altında inceleme: çatlak, lehim kusuru, çizik
- Ölçüm cihazıyla boyut kontrolü: tolerans uygunluğu
- Isıtma ve soğutma: termal genleşme davranışı
- Titreşim uygulaması: bağlantı gevşemesi
- Yük bindirme: esneme ve şekil değişimi bölgelerinin tespiti
- Söküp yeniden takma: montaj hatasına açıklık
- Hatalı montaj denemesi: fiziksel engelleme mevcut mu?
- Kir, nem ve toz gibi çevresel etkilerin simülasyonu
- İç yapının kesitle incelenmesi (destructive inspection / destructive testing)

Süreç Boyunca Sorulan Sorular

- Bu çözüm neden bu biçimde tasarlanmış?
- Buradaki boşluk tasarım gereği mi?
- Kablo neden bu noktada kıvrılıyor?
- Bu vida diğerlerinden neden farklı?
- Teknisyen bu parçayı ters takabilir mi?
- Termal genleşme sonucu bu iki parça temas edebilir mi?

Örnek olarak bir uzay aracı konnektörü ele alınabilir. Teknik çizim üzerinden yapılan inceleme yeterli değildir; gerçek parça elde tutularak “Bu bileşen 180° döndürüldüğünde de takılabiliyor mu?” sorusu sınanmalıdır. Takılabiliyorsa, doğrudan bir insan hatası riski tespit edilmiş olur.

Örnek 1: Bir Kod Parçasının İrdelenmesi

İncelenen Kod

```
async function transfer(from, to, amount) {
  const balance = await getBalance(from);

  if (balance >= amount) {
    await debit(from, amount);
    await credit(to, amount);
    return true;
  }

  return false;
}
```

Yüzey Düzeyi Kontrol

`amount` negatif olabilir mi? `from === to` durumu mümkün mü? `from` veya `to` tanımsız geldiğinde davranış nedir? `amount = NaN` girdisi nasıl işlenir? Parasal değer için kayan noktalı sayı mı kullanılıyor?

Varsayım Katmanı

Kod, `getBalance()` ile `debit()` çağrıları arasında bakiyenin değişmeyeceğini varsaymaktadır. Bakiye 100 iken eşzamanlı iki isteğin her biri 80 birim transfer talep ederse, her ikisi de bakiyeyi 100 olarak okuyup koşul kontrolünden geçebilir. Tespit edilen kusur: **race condition**.

İşlem Bütünlüğü Katmanı

`debit` başarılı, `credit` başarısız olduğunda tutar gönderen hesaptan düşer ancak alıcı hesaba ulaşmaz. Tespit edilen kusur: **atomicity** ihlali.

Dış Sistem Katmanı

`credit()` çağrısı harici bir servise HTTP isteği gönderiyorsa ve yanıt alınmadan bağlantı kesilirse, karşı sistem işlemi tamamlamış olabilir ancak çağıran sistem bu bilgiden yoksundur. Yeniden gönderim hâlinde çift kayıt oluşur. Tespit edilen kusur: **idempotency** eksikliği.

Zaman Katmanı

```
getBalance -> 12 ms
debit      -> 800 ms
credit     -> timeout
```

Kontrollü test ortamında sorunsuz çalışan sistem, gerçek ağ koşullarında farklı davranış sergiler.

Gözlemlenebilirlik Katmanı

İşlem başarısız olduğunda nedeni sonradan tespit edilebiliyor mu? Transaction ID, log kaydı, hesap bilgisi, servis adı, retry sayısı ve sürüm bilgisi mevcut mu? Aksi hâlde sistem hata üretmiş ancak bu hatanın tanısısal değerini yitirmiş olur.

Güvenlik ve Gereksinim Katmanları

from hesabının isteği yapan kullanıcıya ait olduğu hangi mekanizmayla doğrulanmıştır? Ayrıca “transfer” teriminin operasyonel tanımı nedir: gönderen hesaptan düşülmesi mi, alıcıya ulaşması mı, her iki işlemin tek bir transaction içinde gerçekleşmesi mi? Kullanıcıya başarı yanıtı döndüğünde hangi garanti verilmiş olmaktadır?

Örnek 2: Bir Entegre Devrenin İrdelenmesi

Görsel İnceleme

Paket üzerinde çatlak mevcut mu? Pin geometrisi bozulmuş mu? Lehim kalitesi yeterli mi? Flux kalıntısı, yanık izi veya PCB üzerinde anormal renk değişimi gözleniyor mu?

Geometri Kontrolü

Bileşen doğru yönde monte edilmiş mi? Pin 1 konumu doğru mu? Footprint uygun mu? Pad ölçüleri doğru mu? İzler arası mesafe yeterli mi?

Bağlantı Doğrulaması

- VCC gerilimi gerçekten 3.3 V seviyesinde mi?
- GND fiilen referans toprak mı?
- RESET hattı floating durumda mı?
- CLOCK sinyali ulaşıyor mu?

Güç Tüketimi

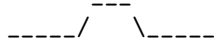
Beklenen değerler idle durumda 10 mA, yük altında 80 mA iken ölçülen değer 170 mA ise, akım tüketimi görünmeyen bir arızanın göstergesi olarak değerlendirilmelidir.

Termal Davranış

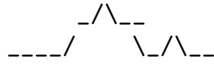
Normal çalışma sırasında lokal bir bölgede aşırı ısınma gözleniyor mu? Termal harita, arızanın konumsal tespitine imkân verir.

Sinyal Bütünlüğü

Mantıksal düzeyde 1/0 gözlemi yeterli değildir. Temiz bir hat profili:



Bozulmuş bir hat profili:



Overshoot, ringing, jitter ve gürültü bileşenleri nedeniyle mantıksal olarak doğru okunan bir sinyal, fiziksel düzeyde kabul edilemez nitelikte olabilir.

Zamanlama Analizi

Clock sinyalinin varlığı tek başına yeterli değildir; verinin clock’a göre doğru fazda gelip gelmediği sınanmalıdır. Bu inceleme setup time, hold time ve propagation delay kavramlarını gündeme getirir.

Çevresel Koşul Değişimi

Düşük ve yüksek sıcaklıkta, besleme geriliminin düşmesi hâlinde ve clock frekansının artırılması durumunda davranış nasıl değişmektedir? Nominal çalışma noktasında gizli kalan problemler, sistemin sınır koşullarına sürülmesiyle açığa çıkarılır.

Üretim Ölçeği

Prototip düzeyinde çalışan tasarımın 10.000 adetlik üretimde 9.998 birimde çalışıp 2 birimde çalışmaması, tolerans dağılımlarının incelenmesini gerektirir.

Tasarım Gerekçelerinin Sorgulanması

Bu pin neden pull-up dirence bağlanmış? Direnç değeri neden 10 k Ω ? Bu konumda kondansatör neden yer alıyor? Trace uzunluğu neden bu ölçüde? Bu voltaj regülatörü hangi gerekçeyle seçilmiş? Her fiziksel tasarım kararı için “neden” sorusu yöneltilmelidir.

Yazılım ve Donanım Arasındaki Yapısal Paralellik

İrdelemenin temel hareketi her iki alanda özdeştir:

1. Nesneyi bir katman aşağı indir
2. Varsayımlarını açığa çıkar
3. Sınırlarını zorla
4. Arayüzlerini incele
5. Zamanlamayı değiştir
6. Kontrollü hata enjekte et
7. Ölç
8. Anormallik ara
9. Bir katman yukarı çıkarak sonucun sistem düzeyindeki etkisini sorgula

Code review, debugging, failure analysis, chip validation, design review ve reverse engineering farklı adlandırmalara sahip olsalar da hepsi bu ortak hareketi içerir.

İrdeleme Prosedürü

Evrensel nitelikte tek bir standart mevcut değildir; ancak design review, verification & validation, FMEA, fault-tree analysis, hazard analysis, root-cause analysis, red teaming ve test & evaluation gibi yerleşik yöntemsel çerçeveler bulunmaktadır. Bunların ortak özü aşağıdaki sıraya indirgenebilir:

1. **Nesneyi tanımla.** İnceleme konusu nedir? Kapsam sınırı nerede başlayıp nerede bitmektedir?
2. **Amacı belirle.** Sistemin işlevi nedir? Başarılı sayılması için hangi koşulları sağlaması gerekir?
3. **Ayrıştır.** Yazılımda: sistem → servis → modül → fonksiyon → satır. Donanımda: sistem → kart → çip → blok → devre.
4. **Arayüzleri çıkar.** Girdi ve çıktılar nelerdir? Hangi bileşen hangisiyle iletişim kurmaktadır? Güç, veri, mekanik yük ve insan etkileşimi hangi noktalardan geçmektedir?
5. **Varsayımları belgelendir.** Sabit kabul edilen her önerme açık hâle getirilmelidir.
6. **Nominal senaryoyu çalıştır.** Sistem beklenen koşullarda gerçekten işlev görüyor mu?
7. **Sınır koşullarını sına.** Minimum, maksimum, sıfır, aşırı büyük, aşırı küçük, hızlı, yavaş, sıcak, soğuk ve yoğun yük koşulları.
8. **Hata enjekte et.** Sensör devre dışı kalırsa? Paket kaybolursa? Bellek dolarsa? Kablo ayrılırsa? Bozuk veri gelirse?
9. **Zamanlamayı değiştir.** Aynı olay erken, geç veya eşzamanlı gerçekleşirse sonuç ne olur?
10. **Kombinasyonları sına.** İki küçük hatanın birlikte gerçekleşmesi? İnsan hatası ile donanım hatasının bileşimi? Yük ile gecikmenin çakışması?
11. **Kanıt talep et.** Test sonucu, ölçüm kaydı, analiz raporu veya simülasyon çıktısı nerededir?
12. **Anormallikleri kayıt altına al.** Yalnızca kritik hatalar değil; anormal ses, gecikme, beklenmeyen log kaydı ve küçük ölçekli sıcaklık artışı dâhil.
13. **Kök nedene in.** Soru “ne oldu?” değil, “bunun gerçekleşmesi nasıl mümkün oldu?” biçiminde kurulmalıdır. Bu aşamada 5 Whys, fault tree ve fishbone yöntemleri devreye girer.
14. **Düzeltil ve yeniden zorla.** Düzeltme sonrası aynı test yinelenmeli, müdahalenin yeni kusur üretip üretmediği doğrulanmalıdır.
15. **Dersi genelleştir.** Bu olaydan başka projelere aktarılabilecek hangi ilke türetilmektedir?

Genelleştirme Örneği

Bir kablonun ters takılması durumunda çıkarılacak sonuç “teknisyen hatalı montaj yapmıştır” biçiminde olmamalıdır. Daha yüksek düzeyde bir ders şu şekilde formüle edilir: kritik bağlantılar, hatalı montaja fiziksel olarak imkân vermeyecek biçimde tasarlanmalıdır. Tekil olaydan genel mühendislik ilkesine geçiş tam bu noktada gerçekleşir.

Yöntemlerin İşlevsel Ayrımı

- **FMEA:** “Neler bozulabilir?” sorusunu sistematikleştirir
- **Fault tree analysis:** “Bu istenmeyen sonuç hangi yollardan ortaya çıkabilir?” sorusunu ele alır
- **Verification & Validation:** “İddiamız gerçekten doğru mu?” sorusunu yanıtlar
- **Red teaming:** “Karşı perspektiften bakıldığında sistem nasıl kırılır?” sorusunu yapılandırır

Kavramın Diller Arası Karşılıkları

Söz konusu edim hem fiziksel hem zihinsel bir nitelik taşıdığından, tek bir terimle tam olarak karşılanmamakta; bağlama göre farklı sözcükler kullanılmaktadır.

İngilizce Karşılıklar

Türkçe anlam	İngilizce	Nüans
İnce eleyip sık dokumak	Scrutinize	Mühendislik ve akademide en yaygın terim
Derinlemesine incelemek	Examine thoroughly	Genel kullanım
Kurcalamak	Probe	Yüzeyin altını araştırma
Parçalara ayırıp incelemek	Dissect	Anatomi/cerrahi metaforu
Ayrıntısıyla ayırtırmak	Pick apart	Eleştirel parçalama
İrdelemek	Analyze	Sistematik analiz
Sorgulamak	Question / Challenge	Varsayımları hedef alır
Araştırmak	Investigate	Soruşturmacı yaklaşım
Ayrıntıyı denetlemek	Inspect	Fiziksel muayene
Zayıf noktaları aramak	Stress-test	Dayanıklılık sınaması
Hata aramak	Debug	Yazılma özgü
Tersine çözmek	Reverse engineer	Tasarım mantığının çözümlenmesi

İrdeleme Döngüsünün Otomatikleştirilmesi

Doyum kriteri (saturation) esasen bir döngü koşuludur; bu koşulun insan tarafından elle yürütülmesi sürdürülebilir değildir. Çözüm, döngü mantığının araca veya koda taşınmasıdır. Aşağıda artan olgunluk düzeyine göre dört katman sunulmaktadır.

Katman 0: İstem İçi Otomatikleştirme

En düşük maliyetli yaklaşım, tur mantığının tek bir istem içine gömülmesidir.

Aşağıdaki nesneyi irdele. Turları KENDİN yürüt.

Tur 1: Bulguları üret.
Tur 2: Tur 1'i görmezden gelerek sıfırdan tekrar üret.
Tur 3: Tur 2'yi görmezden gelerek sıfırdan tekrar üret.
...

Her turdan sonra kendine sor:

"Bu turda önceki turlarda OLMAYAN kritik bir bulgu çıktı mı?"

Cevap 3 tur üst üste HAYIR ise dur ve şunu yaz:

DOYUM ULAŞILDI - tur sayısı: N

Maksimum 10 tur. Son çıktı: tüm turların birleşik, tekilleştirilmiş bulgu listesi + tur bazında yeni bulgu sayısı.

Yöntemin sınırı şudur: tek bağlam içinde model kendi önceki turlarına demir atar; sıfırdan başlama gerçekte sağlanmaz ve doyum yapay olarak erken gelir. Buna rağmen tamamen manuel yürütmeye kıyasla önemli kazanım sağlar.

Katman 1: Dosya Tabanlı Döngü

Bağlam kirlenmesini kırmanın pratik yolu, her turun ayrı oturumda çalıştırılması ve ortak hafızanın dosyaya taşınmasıdır.

```
findings.md    <- birikimli bulgu havuzu  
round_log.md   <- tur bazında yeni bulgu sayısı
```

Tur istemi:

1. findings.md'yi OKUMA. Nesneyi sıfırdan irdele.
2. Bulgularını yaz.
3. Şimdi findings.md'yi oku, kendi bulgularınla karşılaştı.
4. Yeni olanları findings.md'ye ekle (tekilleştirerek).
5. round_log.md'ye şunu ekle:
Tur N | yeni kritik: X | yeni önemli: Y | toplam: Z
6. Son 3 turda yeni kritik = 0 ise "DOYUM" yaz.

Bu akış bir slash command veya skill hâline getirildiğinde tek komutla çalışır; kullanıcı müdahalesi yalnızca devam onayına indirgenir.

Katman 2: Betikle Tam Otomatik Döngü

Asıl çözüm budur ve yaklaşık altmış satırlık bir betikle gerçekleştirilebilir.

```
import json, subprocess, hashlib

def run_round(target, round_no):
    # Her tur TEMİZ bağlam - önceki bulgular verilmez
    prompt = SCRUTINY_PROMPT.format(target=target)
    out = call_llm(prompt, temperature=0.9) # çeşitlilik için yüksek
    return json.loads(out) # şema zorunlu

def canonical(finding):
    # Aynı bulgunun farklı ifadelerini eşleştir
    return normalize(finding["location"], finding["failure_mode"])

seen = {}
no_new_critical = 0
log = []

for r in range(1, MAX_ROUNDS + 1):
    findings = run_round(target, r)
    new_critical = 0
    for f in findings:
        key = canonical(f)
        if key not in seen:
            seen[key] = f
            if f["severity"] == "critical":
                new_critical += 1
    log.append({"round": r, "new_critical": new_critical,
               "total": len(seen)})
    no_new_critical = no_new_critical + 1 if new_critical == 0 else 0
    if no_new_critical >= 3:
        break
```

Kritik tasarım noktaları:

- **Yüksek sıcaklık.** Düşük sıcaklıkta her tur benzer çıktı üretir ve doyum sinyali sahte olur. 0.8–1.0 aralığı yeterli çeşitlilik sağlar.
- **Her turda temiz bağlam.** Önceki bulgular modele verilirse model bunları yineler ve yeni bulgu yok sinyali anlamını yitirir. Tekilleştirme modelin değil kodun sorumluluğunda olmalıdır.

- **Kod tarafında tekilleştirme.** Basit dizgi eşleşmesi yetersizdir; aynı arıza farklı ifadelerle anlatılabilir. İki seçenek mevcuttur: gömme (embedding) benzerlik eşiği veya ayrı bir model çağrısıyla eşdeğerlik sorgusu. İkincisi daha doğru, buna karşılık daha maliyetlidir.
- **Katman rotasyonu.** Her tura farklı bir inceleme katmanı atanmalıdır (varsayımlar, zamanlama, insan hatası vb.). Aksi hâlde model aynı katmanı yineleyerek tarar ve doyum, kapsam doyumunu değil yalnızca tekrar doyumunu olur.

Katman 3: Doyum Kriterinin Sertleştirilmesi

Üç tur üst üste yeni kritik bulgu yok ölçütü tek başına kırılıktır. Daha sağlam bileşimler:

Kriter	Yakaladığı durum
3 tur yeni kritik = 0	Klasik doyum
Tüm katmanlar en az 2 kez tarandı	Kapsam doyumunu
Yeni bulgu oranı % 10 altında	Marjinal getiri düşüşü
Yeni bulguların tamamı küçük	Ciddiyet doyumunu
Bütçe (tur / token / süre) tükendi	Sert durdurucu

Uygulamada kapsam koşulu, doyum koşulu ve bütçe tavanı birlikte tanımlanmalıdır: ilk ikisi sağlanmadan döngü durdurulmamalı, üçüncüsü aşıldığında ise zorunlu olarak durdurulmalıdır.

Katman 4: Doyum Eğrisinin Görselleştirilmesi

Tur:	1	2	3	4	5	6	7	8
Yeni krt:	7	4	3	1	2	0	0	0
Toplam:	7	11	14	15	17	17	17	17
							' - doyum	

Bu eğri modelin değil, kullanılan istemin tükenme noktasını gösterir. Eğrinin üçüncü turda düzleşmesi istemin dar olduğuna; on beşinci turda hâlâ bulgu üretilmesi ya nesnenin gerçekten karmaşık olduğuna ya da modelin üretim yaptığına işaret eder.

Uyarı: Doyum Doğruluk Anlamına Gelmez

Otomatik döngünün başlıca riski, doyumun kalite ölçütü sanılmasıdır. Dar bir çerçeveye sıkışmış model hızla doyar ve süreci tamamlanmış olarak raporlar. Karşı önlemler:

- Nesneye bilinen bir kusur kasten yerleştirilir; döngü bu kusuru bulamıyorsa döngü hatalıdır (mutation testing mantığı)
- Aynı görev iki farklı istem ailesiyle çalıştırılır ve doyum noktaları karşılaştırılır
- Doyum sonrası tek bir hangi katman hiç incelenmedi? turu çalıştırılır

Her hâlükârda çıktı bir hipotez listesidir; ciddiyet ataması modelin kendi tahmini olup doğrulanmış bulgu niteliği taşımaz.

Uygulama Sıralaması

1. Katman 0 isteminin bir proje talimatına veya skill'e yerleştirilmesi — kod gerektirmez, manuel tekrar önemli ölçüde azaltır
2. Katman 1 dosya tabanlı döngünün slash command olarak kurulması
3. İhtiyaç sürüyorsa Katman 2 betiğinin yazılması — temiz bağlam ve kod tarafında tekilleştirme ancak bu düzeyde elde edilir

Kavramın Tarihsel Soykütüğü

Bu pratiğin tek bir mucidi bulunmamaktadır. Birbirinden bağımsız doğmuş dört-beş damarın 20. yüzyılda birleşmesiyle bugünkü hâlini almıştır.

Felsefi Kök: Sokrates

En erken biçimi *elenkhos*, yani çapraz sorgulama yöntemidir. Sokrates muhatabının tezini kabul eder, o tezdten çıkan sonuçları tek tek türetir ve çelişkiye ulaştırır. Yöntemin özü bugünküyle örtüşür: iddiayı savunmak değil, dayanamayacağı noktayı bulmak.

Bilimsel Kök: Bacon ve Popper

Francis Bacon (1620, *Novum Organum*): doğaya soru sormak yeterli değildir; doğanın kontrollü koşullarda zorlanması gerekir. Deney kavramının bu biçimi, sınır koşullarını yokla adınının atasıdır. Bacon ayrıca insan bilişindeki sistem-atik kör noktaları sınıflandırmıştır (*idola*: kabile, mağara, çarşı ve tiyatro putları); bu, meta-review katmanının ilk formülasyonu sayılabilir.

Karl Popper (1934, *Logik der Forschung*): asıl kırılma noktasıdır. Bir teoriyi doğrulayan gözlem toplamak düşük değerlidir; değerli olan, teoriyi yanlışlayabilecek testin aranmasıdır. Yanlışlanabilirlik ilkesi, bu pratiğin epistemolojik temelini oluşturur.

Imre Lakatos yaklaşımı bir adım ileri taşımıştır: teoriler tek tek değil, araştırma programları hâlinde savunulur ve programın çekirdeğini koruyan bir yardımcı hipotezler kuşağı bulunur. Hangi varsayım bozulursa tasarım çöker? sorusu tam olarak bu yapıyı hedef alır.

Mühendislik Kökü: 1940–1970

Bu dönemde pratik kurumsallaşmış ve bugün kullanılan yöntem adları doğmuştur.

Tarih	Yöntem	Kaynak
1949	FMEA	ABD ordusu, MIL-P-1629
1950'ler	Fishbone / Ishikawa	Kaoru Ishikawa, Kawasaki
1950'ler	5 Whys	Sakichi Toyoda / Taiichi Ohno, Toyota
1962	Fault Tree Analysis	H. A. Watson, Bell Labs (Minuteman)
1960'lar	HAZOP	ICI, kimya endüstrisi
1970'ler	Design Review / V&V	NASA, savunma sanayii
1976	Software inspection	Michael Fagan, IBM

Dikkat çekici nokta şudur: bu yöntemlerin hemen tamamı bir felaketin ardından doğmuştur. İrdeleme kültürü kuramdan değil, enkazdan türemiştir.

Kurumsal Kök: NASA ve Feynman

Richard Feynman'ın 1986 tarihli Challenger raporu eki, mühendislik irdemesinin en bilinen tekil metnidir. Yönetimin beyan ettiği arıza olasılığı

ile mühendislerin tahmini arasında binlerce kat fark tespit etmiş ve raporunu, gerçekliğin halkla ilişkilerden önce geldiği yönündeki tespitle bitirmiştir.

Bu örnek, irdelemenin yalnızca teknik değil kurumsal bir edim olduğunu göstermektedir: sistemin kendisi kadar, sistem hakkında konuşma biçimi de incelenmelidir. Failure analysis disiplininin ayrı bir alan olarak kurumsallaşması da aynı döneme denk gelir.

Yazılım Kolu

- **Fagan inspection** (1976): kod incelemesinin ilk formel süreci
- **Fuzzing** (1988, Barton Miller, Wisconsin): rastgele girdiyle programı kırma; hata enjekte et adımının otomatikleşmiş biçimi
- **Chaos Engineering** (2011, Netflix Chaos Monkey): üretim ortamında kasıtlı arıza üretimi
- **Postmortem kültürü** (Google SRE): suçlayıcı olmayan kök neden analizi

Güvenlik Kolu

Red teaming terimi Soğuk Savaş dönemi askerî tatbikatlarından gelmektedir (kırmızı renk karşı tarafı temsil eder). RAND Corporation bünyesinde savunma senaryolarının düşman perspektifinden sınanması pratiği, sonradan siber güvenliğe ve oradan yapay zekâ değerlendirmesine aktarılmıştır.

Bilişsel Kol

- **Charlie Munger — inversion:** problemi tersten kurma; nasıl başarılı olurum yerine nasıl başarısız olurum sorusu
- **Gary Klein — premortem:** proje başlamadan önce bir yıl sonra bu proje başarısız oldu, nedenini yazınız egzersizi
- **Kahneman ve Tversky:** sistematik bilişsel yanlılıklar; irdeleme ihtiyacının psikolojik gerekçesi
- **Irving Janis (1972) — groupthink:** ekip düzeyindeki kör noktanın kavramsallaştırılması

Terminolojik Not

Türkçedeki didiklemek fiilinin bir mühendislik terimi olarak önerildiği bilinen bir kaynak bulunmamaktadır; konuşma dilinden alınmış bir mecazdır. Literatürde yerleşik karşılığı yoktur; akademik metinde *scrutiny*, *irdeme* veya *derinlemesine inceleme* tercih edilir.

Özet Zincir

Sokrates (sorgulama)

- > Bacon (kontrollü zorlama + kör noktalar)
- > Popper (yanlışlanabilirlik)
- > Toyota / Bell Labs / NASA (yöntemleşme)
- > Feynman (kurumsal irdeleme)
- > Fagan / Miller / Netflix (yazılıma aktarım)
- > Klein / Munger (bilişsel tersine çevirme)

Tüm damarlarda ortak olan ilke aynıdır: onaylayıcı kanıt toplamak ucuz ve yanıltıcıdır; değerli olan, iddiayı kırmayı denemek ve bunda başarısız olmaktır.

Kaynak Doğrulama Notu

Yukarıdaki tarih ve atıfların bir kısmı ikincil hafızaya dayanmaktadır. Özellikle FMEA'nın MIL-P-1629 tarihlemesi ve Watson'ın Fault Tree Analysis çalışması, akademik kullanım öncesinde birincil kaynaktan doğrulanmalıdır.

Diğer Dillerdeki Karşılıklar

- **Almanca — Fehlersuche:** Sözcük anlamı “hata arama”. Alman mühendislik geleneğinde yerleşik bir kavramdır.
- **Fransızca — Décortiquer:** Sözcük anlamı “kabuğunu soyup içine bakmak”.
- **İtalyanca — Smontare:** Sözcük anlamı “sökme”; mecazi olarak bir tezi madde madde çürütmek anlamında da kullanılır.
- **İspanyolca — Desmenuzar:** Sözcük anlamı “ufalamak”; bir düşünceyi en küçük bileşenlerine ayırarak inceleme.